

P1144R12

std::is_trivially_relocatable

Published Proposal, 2024-10-15

Author:

[Arthur O'Dwyer](#)

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

P1144 trivial relocation is used by Abseil, AMC, BSL, Folly, HPX, Parlay, Pocketpy, Qt, Subspace, and Thrust. Let's make it part of the C++ standard.

Table of Contents

- 1 Changelog**
- 2 What's going on here?**
- 3 Proposed wording**
 - 3.1 [cpp.predefined]
 - 3.2 [version.syn]
 - 3.3 [defns.relocation]
 - 3.4 [basic.types.general]
 - 3.5 [class.prop]
 - 3.6 [dcl.attr.trivreloc]
 - 3.7 [cpp.cond]
 - 3.8 [concepts.syn]

- 3.9 [concept.relocatable]
- 3.10 [memory.syn]
- 3.11 [meta.type.synop]
- 3.12 [meta.unary.prop]
- 3.13 [algorithms.requirements]
- 3.14 [uninitialized.relocate]
- 3.15 [specialized.relocate]

4 Acknowledgements

Appendix A: Straw polls

Poll taken in EWG at St Louis on 2024-06-28

Polls taken in EWGI at Issaquah on 2023-02-10

Polls taken in EWGI at Prague on 2020-02-13

References

Informative References

§ 1. Changelog

- R12 (pre-Wrocław 2024):
 - Complete rewrite of the prose sections.
 - Added missing forward-declarations to [concepts.syn], [meta.type.synop], and [memory.syn].
 - Removed `[[nodiscard]]` from `relocate(T*)`, thanks to [P2422](#).
 - Updated [concept.relocatable]: `u2` mustn't be a potentially overlapping subobject.
 - Updated [uninitialized.relocate] and [specialized.relocate]: Instead of "side effects *might* not happen," we now say "*do* not happen": Memmove optimization is now mandatory. (This addresses one of EWG's concerns versus [P2786](#), that P1144 leaves too much up to quality-of-implementation.) This incidentally guarantees that `relocate` of a trivially relocatable type will never throw.

§ 2. What's going on here?

Isn't EWG already considering a "trivial relocation" proposal, [\[P2786\]](#)?

Yes. P2786 was forwarded (1–8–3–3–1) from EWGI to EWG at Issaquah in February 2023.

P1144, which dates back to 2018, had *already* been voted out of EWGI once, at Prague in February 2020 (1–3–4–1–0). It remains unclear why that vote was ignored. At Issaquah, it was seen and voted *again* (0–7–4–3–1).

At Tokyo in February 2024, P2786 was voted from EWG into CWG (7–9–6–0–2). But then, at St Louis in June 2024, EWG voted strongly (21–15–3–6–5) that P2786 wasn't ready for CWG after all, and brought it back for further discussion. I [asked](#) during that meeting whether the chair of EWG was willing to schedule P1144 for discussion, or even to take a poll of EWG on *whether* to discuss it. He said no.

That seems... weird. Is P2786 simply better?

In my opinion, no it is not. It's more complicated and offers less to the programmer. It has had no uptake in the programming community. In fact, the main reason it was so overwhelmingly clawed back in St Louis is because of two papers presented there by third-party library maintainers:

- [\[P3233\]](#) "Issues with P2786" (Giuseppe D'Angelo)
- [\[P3236\]](#) "Please reject P2786 and adopt P1144" (Stéphane Janel, Hans Goudey, Jacques Lucke, Alan de Freitas, Krystian Stasiowski, Daniel Liam Anderson, Giuseppe Ottaviano, Giuseppe D'Angelo, Thiago Maciera, Hartmut Kaiser, Isidoros Tsaousis, Shreyas Atre)

Does P1144 have uptake in the community?

Yes. In fact, since P1144R0 was modeled directly on Folly and BSL, you might say that P1144 is the *committee's* uptake of a *community* idea!

Since 2018, most of the libraries that inspired P1144 have entrenched even further with its consistent terminology and semantics. P1144R9 introduced a pair of feature-test macros, and some libraries have adopted those macros, so that they *already today* get better performance on Arthur's reference implementation and will be "ready on day one" when P1144 is eventually adopted by the committee. The flagship examples are:

- Facebook Folly defines its `folly::IsRelocatable` trait in terms of P1144's `std::is_trivially_relocatable<T>` if `__cpp_lib_trivially_relocatable` is defined. Otherwise, Folly falls back on `is_trivially_copyable`.

- Stellar HPX defines `hpx::is_trivially_relocatable` in terms of P1144's `std::is_trivially_relocatable<T>` if `HPX_HAVE_P1144_RELOCATE_AT` is defined at build time.
- Google Abseil defines `absl::is_trivially_relocatable` with P1144 semantics. Abseil will directly use Clang's `__is_trivially_relocatable(T)` builtin, but not when compiled with Clang trunk — only if `__cpp_impl_trivially_relocatable` is defined. ([Godbolt.](#))
- Carnegie Mellon's ParlayLib project defines `parlay::is_trivially_relocatable` in terms of P1144's `std::is_trivially_relocatable<T>` if `__cpp_lib_trivially_relocatable` is defined. Otherwise, Parlay falls back on the Clang builtin.

Wait. Clang has a builtin trait for this?

Yes. That builtin was added to trunk by Devin Jeanpierre [in February 2022](#), about a year before P2786 was released to the public. The commit message indicated that "rather than trying to pick a winning proposal for trivial relocation operations" Clang would implement its own semantics, which turned out to be very similar to the semantics chosen by P2786 the next year.

Sadly, Clang's chosen semantics have proven useless to many existing libraries. Folly, Abseil, HPX, and Subspace all considered using Clang's builtin and decided against it, due to its current semantics. Giuseppe D'Angelo, on the KDAB company blog, [observes](#):

P2786's design is limiting and not user-friendly, to the point that there have been serious concerns that existing libraries may not make use of it at all. In particular, P2786's relocation semantics do not match Qt's. With my Qt hat on, *it soon became clear that we could not have used P2786 in Qt to replace our own implementation of trivial relocation.*

[There was an effort in March 2024](#) to give Clang trunk's `__is_trivially_relocatable(T)` P1144 semantics. The pull request was supported with comments by maintainers of Abseil, Qt, AMC, Subspace, HPX, and several smaller libraries as well. As of this writing (October 2024), the pull request has not been accepted. These libraries continue to rely on P1144's feature-test macros (or just use `is_trivially_copyable`), because Clang trunk doesn't support their use-cases.

And everyone just uses `memcpy` instead of `std::uninitialized_relocate`, right?

Right. Many of them have implemented a P1144-alike API in *terms* of `memcpy`, though:

- Amadeus AMC's `amc::uninitialized_relocate`, `uninitialized_relocate_n`, and `relocate_at`

- HPX's `hpx::experimental::uninitialized_relocate`, `uninitialized_relocate_n`, `uninitialized_relocate_backward`, and `relocate_at`
- Parlay's `uninitialized_relocate` and `uninitialized_relocate_n`
- Qt's `q_uninitialized_relocate_n`

Could these two halves of the feature be severed?

You mean, ship just the `is_trivially_relocatable<T>` trait in C++26, and then worry about the algorithms like `uninitialized_relocate` (and the `trivially_relocatable` attribute itself) for C++29? Sure, and I'd be fine with that — as long as the standard trait were defined in the P1144 way, so that all these libraries can start using it.

Hasn't P2786 adjusted its trait in that direction recently?

Yes. Following EWG's clawback in St Louis 2024, P2786R7 decided that its `is_replaceable` trait should consider the move-assignment operator as well as the move-constructor. That's a positive step. But, as of this writing, P2786 still considers unannotated types "eligible for trivial relocation" when they have a non-defaulted copy-constructor — whereas P1144 conservatively will not *assume* that any type is trivially relocatable unless it is well-understood by the compiler (i.e., Rule of Zero). Also, P2786 considers unannotated types "eligible for trivial relocation" even if they are polymorphic; see [\[Polymorphic\]](#) for how that can cause segfaults in supposedly "well-defined" code.

Beyond the criteria for relocatability, P2786 has other unacceptable downsides. It bifurcates the P1144 trait into two traits — `is_trivially_relocatable` and `is_replaceable`, where in fact the *latter* corresponds to what everyone names `is_trivially_relocatable` today. That will lead to confusion. P2786 invents new functions like `swap_value_representations` and `trivially_relocate`, again not reflecting existing practice. `swap_value_representations` is defined to magically omit copying vptrs (even of subobjects), which means different codegen for different "replaceable" types of the same size and alignment. Contrariwise, P1144 allows us to share the same code for all trivially relocatable types of the same size, as seen in [this Godbolt](#). That's basically the point of calling the code "trivial"!

It is unambiguously a good thing every time P2786 takes a step toward P1144. But many of those steps remain to be taken.

And EWG still refuses to directly discuss P1144 itself?

So far, yes. I remain hopeful that one day that may change.

Where can I learn more about trivial relocation?

Besides [Arthur's blog](#), you might also want to look at:

- "[Trivially relocatable types in C++/Subspace](#)", by Dana Jansens
- "[Qt and trivial relocation](#)", by Giuseppe D'Angelo, on the KDAB blog
- "[Relocation semantics in the HPX library](#)", by Isidoros Tsaousis
- "[Object relocation](#)" in the Folly repository

§ 3. Proposed wording

The wording in this section is relative to the current working draft.

§ 3.1. [\[cpp.predefined\]](#)

Add a feature-test macro to the table in [\[cpp.predefined\]](#):

```
__cpp_impl_three_way_comparison    201907L
__cpp_impl_trivially_relocatable   YYYYMMML
__cpp_implicit_move                202207L
```

§ 3.2. [\[version.syn\]](#)

Add a feature-test macro to [\[version.syn\]/2](#):

```
#define __cpp_lib_transparent_operators    201510L // freestanding, also in <memory>, <f
#define __cpp_lib_trivially_relocatable   YYYYMMML // freestanding, also in <memory>, <t
#define __cpp_lib_tuple_element_t         201402L // freestanding, also in <tuple>
```

§ 3.3. [\[defs.relocation\]](#)

Add a new section in [\[intro.defs\]](#):

relocation operation [\[defns.relocation\]](#)

the homogeneous binary operation performed by `std::relocate` at, consisting of a move construction immediately followed by a destruction of the source object

§ 3.4. [\[basic.types.general\]](#)

Add a new section in [\[basic.types.general\]](#):

9. Arithmetic types ([\[basic.fundamental\]](#)), enumeration types, pointer types, pointer-to-member types ([\[basic.compound\]](#)), `std::nullptr_t`, and *cv*-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types ([\[class.prop\]](#)), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, trivial class types ([\[class.prop\]](#)), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivial types*. Scalar types, standard-layout class types ([\[class.prop\]](#)), arrays of such types, and *cv*-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types ([\[class.prop\]](#)), array types, and *cv*-qualified versions of these types are collectively called *implicit-lifetime types*.

x. Trivially copyable types, trivially relocatable class types ([\[class.prop\]](#)), arrays of such types, and *cv*-qualified versions of these types are collectively called *trivially relocatable types*.

[Note: For a trivially relocatable type, the relocation operation ([\[defns.relocation\]](#)) as performed by, for example, `std::swap` ranges or `std::vector::reserve`, is tantamount to a simple copy of the underlying bytes. —end note]

[Note: It is likely that many standard library types are trivially relocatable types. —end note]

10. A type is a *literal type* if it is: [...]

§ 3.5. [\[class.prop\]](#)

DRAFTING NOTE: For the "if supported" wording, compare [\[dcl.attr.nouniqueaddr\]/2](#) and [\[cpp.cond\]/5](#).

DRAFTING NOTE: The wording proposed here deliberately echoes the existing "trivially copyable" wording. "Copyable" and "relocatable" are siblings; every trivially copyable type is trivially relocatable by definition. However, CWG already knows the wording for "trivially copyable" does not match library-writers' expectations; see [\[P3279\]](#) for examples and a possible direction. If that direction is taken, then we'd certainly update the proposed wording of "trivially relocatable" to follow the new wording of "trivially copyable."

Modify [\[class.prop\]](#) as follows:

1. A *trivially copyable class* is a class:

- that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator ([\[special\]](#), [\[class.copy.ctor\]](#), [\[class.copy.assign\]](#)),
- where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- that has a trivial, non-deleted destructor ([\[class.dtor\]](#)).

2. A *trivial class* is a class that is trivially copyable and has one or more eligible default constructors ([\[class.default.ctor\]](#)), all of which are trivial. [*Note:* In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. — *end note*]

x. A *trivially relocatable class* is a class:

- where no eligible copy constructor, move constructor, copy assignment operator, move assignment operator, or destructor is user-provided,
- which has no virtual member functions or virtual base classes,
- all of whose non-static data members are either of reference type or of trivially relocatable type ([\[basic.types.general\]](#)), and
- all of whose base classes are of trivially relocatable type;

or a class that is declared with a `trivially_relocatable` attribute with value `true` ([\[dcl.attr.trivrelloc\]](#)) if that attribute is supported by the implementation ([\[cpp.cond\]](#)).

3. A class *S* is a *standard-layout class* if it: [...]

§ 3.6. [\[dcl.attr.trivrelloc\]](#)

DRAFTING NOTE: For the "Recommended practice" wording, compare [\[dcl.attr.nouniqueaddr\]/2](#).

Add a new section after [\[dcl.attr.nouniqueaddr\]](#):

1. The *attribute-token* `trivially_relocatable` specifies that a class type's relocation operation has no visible side-effects other than a copy of the underlying bytes, as if by the library function `std::memcpy`. It may be applied to the definition of a class. It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, shall have the form

(*constant-expression*)

The *constant-expression* shall be an integral constant expression of type `bool`. If no *attribute-argument-clause* is present, it has the same effect as an *attribute-argument-clause* of `(true)`.

2. If any definition of a class type has a `trivially_relocatable` attribute with value *V*, then each definition of the same class type shall have a `trivially_relocatable` attribute with value *V*. No diagnostic is required if definitions in different translation units have mismatched `trivially_relocatable` attributes.

3. If a class type is declared with the `trivially_relocatable` attribute, and the program relies on observable side-effects of its relocation other than a copy of the underlying bytes, the behavior is undefined.

4. *Recommended practice:* The value of a *has-attribute-expression* for the `trivially_relocatable` attribute should be `0` for a given implementation unless this attribute can cause a class type to be trivially relocatable (`[class.prop]`).

§ 3.7. [\[cpp.cond\]](#)

Add a new entry to the table of supported attributes in [\[cpp.cond\]](#):

<code>noreturn</code>	200809L
<code>trivially_relocatable</code>	YYYYMML
<code>unlikely</code>	201803L

§ 3.8. [\[concepts.syn\]](#)

Modify [\[concepts.syn\]](#) as follows:

```
// [concept.copyconstructible], concept copy_constructible
template<class T>
    concept copy_constructible = see below;

// [concept.relocatable], concept relocatable
template<class T>
    concept relocatable = see below;
```

§ 3.9. [concept.relocatable]

DRAFTING NOTE: We intend that a type may be relocatable regardless of whether it is copy-constructible; but, if it is copy-constructible then copy-and-destroy must have the same semantics as move-and-destroy. We intend that a type may be relocatable regardless of whether it is assignable; but, if it is assignable then assignment must have the same semantics as destroy-and-copy or destroy-and-move. The semantic requirements on assignment help us optimize `vector::insert` and `vector::erase`. `pmr::forward_list<int>` satisfies `relocatable`, but it models `relocatable` only when all relevant objects have equal allocators.

Add a new section after `[concept.copyconstructible]`:

`relocatable` concept [concept.relocatable]

```
template<class T>
    concept relocatable = move_constructible<T>;
```

1. If `T` is an object type, then let `rv` be an rvalue of type `T`, `lv` an lvalue of type `T` equal to `rv`, and `u2` a distinct complete object of type `T` equal to `rv`. `T` models `relocatable` only if

- After the definition `T u = rv;`, `u` is equal to `u2`.
- `T(rv)` is equal to `u2`.
- If the expression `u2 = rv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(rv);`
- If the definition `T u = lv;` is well-formed, then after the definition `u` is equal to `u2`.
- If the expression `T(lv)` is well-formed, then the expression's result is equal to `u2`.
- If the expression `u2 = lv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(lv);`

§ 3.10. [memory.syn]

Modify [memory.syn] as follows:

```

template<class InputIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_move(InputIterator first,
                                              InputIterator last,
                                              NoThrowForwardIterator result);

template<class ExecutionPolicy, class ForwardIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec,
                                              ForwardIterator first, ForwardIterator last,
                                              NoThrowForwardIterator result);

template<class InputIterator, class Size, class NoThrowForwardIterator>
    pair<InputIterator, NoThrowForwardIterator>
        uninitialized_move_n(InputIterator first, Size n,
                             NoThrowForwardIterator result);

template<class ExecutionPolicy, class ForwardIterator, class Size,
        class NoThrowForwardIterator>
    pair<ForwardIterator, NoThrowForwardIterator>
        uninitialized_move_n(ExecutionPolicy&& exec,
                             ForwardIterator first, Size n, NoThrowForwardIterator result);

namespace ranges {
    template<class I, class O>
        using uninitialized_move_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S1,
            nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
        requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
        uninitialized_move_result<I, O>
            uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);

    template<input_range IR, nothrow-forward-range OR>
        requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
        uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
            uninitialized_move(IR&& in_range, OR&& out_range);

    template<class I, class O>
        using uninitialized_move_n_result = in_out_result<I, O>;

    template<input_iterator I,
            nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
        requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
        uninitialized_move_n_result<I, O>
            uninitialized_move_n(I ifirst, iter_difference_t<I> n,
                                O ofirst, S olast);
}

template<class NoThrowInputIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_relocate(NoThrowInputIterator first,
                                                  NoThrowInputIterator last,
                                                  NoThrowForwardIterator result);

template<class ExecutionPolicy, class NoThrowForwardIterator1, class NoThrowForwardIterator2>
    NoThrowForwardIterator2 uninitialized_relocate(ExecutionPolicy&& exec,
                                                  NoThrowForwardIterator1 first, NoThrowForwardIterator2 result);

```

```

template<class NoThrowInputIterator, class Size, class NoThrowForwardIterator>
    pair<NoThrowInputIterator, NoThrowForwardIterator>
        uninitialized_relocate_n(NoThrowInputIterator first, Size n,
                                NoThrowForwardIterator result); // fr
template<class ExecutionPolicy, class NoThrowForwardIterator1, class Size,
        class NoThrowForwardIterator2>
    pair<NoThrowForwardIterator1, NoThrowForwardIterator2>
        uninitialized_relocate_n(ExecutionPolicy&& exec, // se
                                NoThrowForwardIterator1 first, Size n, NoThrowForwardItera

template<class NoThrowBidirectionalIterator1, class NoThrowBidirectionalIterator2>
    NoThrowBidirectionalIterator2
        uninitialized_relocate_backward(NoThrowBidirectionalIterator1 first,
                                        NoThrowBidirectionalIterator1 last,
                                        NoThrowBidirectionalIterator2 result); // fr
template<class ExecutionPolicy, class NoThrowBidirectionalIterator1, class NoThrowBidir
    NoThrowBidirectionalIterator2
        uninitialized_relocate_backward(ExecutionPolicy&& exec,
                                        NoThrowBidirectionalIterator1 first,
                                        NoThrowBidirectionalIterator1 last,
                                        NoThrowBidirectionalIterator2 result); // fr

template<class NoThrowForwardIterator, class T>
    void uninitialized_fill(NoThrowForwardIterator first, // fr
                           NoThrowForwardIterator last, const T& x);

[... ]
    template<nothrow-input-iterator I>
        requires destructible<iter_value_t<I>>
            constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept; // fr
}

// [specialized.relocate], relocate
template<class T>
    T *relocate_at(T* source, T* dest); // fr

template<class T>
    remove_cv_t<T> relocate(T* source); // fr

// [unique.ptr], class template unique_ptr

```

§ 3.11. [meta.type.synop]

Modify [meta.type.synop] as follows:

```

[...]
// \[meta.unary.prop\], type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
template<class T> struct is_trivial;
template<class T> struct is_trivially_copyable;
template<class T> struct is\_trivially\_relocatable;
template<class T> struct is_standard_layout;
[...]

```

§ 3.12. [\[meta.unary.prop\]](#)

Add a new entry to Table 47 in [\[meta.unary.prop\]](#):

Template	Condition	Preconditions
template<class T> struct is_trivially_copyable;	T is a trivially copyable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_trivially_relocatable;	T is a trivially relocatable type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.
template<class T> struct is_standard_layout;	T is a standard-layout type ([basic.types.general])	remove_all_extents_t<T> shall be a complete type or cv void.

§ 3.13. [\[algorithms.requirements\]](#)

Modify [\[algorithms.requirements\]](#) as follows:

- If an algorithm's template parameter is named `InputIterator`, `InputIterator1`, or `InputIterator2`, or `NoThrowInputIterator`, the template argument shall meet the *Cpp17InputIterator* requirements ([input.iterators]).
- If an algorithm's template parameter is named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, the template argument shall meet the *Cpp17OutputIterator* requirements ([output.iterators]).
- If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, `ForwardIterator2`, or `NoThrowForwardIterator`, `NoThrowForwardIterator1`, or `NoThrowForwardIterator2`, the template argument shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]) if it is required to be a mutable iterator, or model `forward_iterator` ([iterator.concept.forward]) otherwise.
- If an algorithm's template parameter is named `NoThrowInputIterator`, `NoThrowForwardIterator`, `NoThrowForwardIterator1`, or `NoThrowForwardIterator2`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.
- If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, or `BidirectionalIterator2`, `NoThrowBidirectionalIterator1`, or `NoThrowBidirectionalIterator2`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators]) if it is required to be a mutable iterator, or model `bidirectional_iterator` ([iterator.concept.bidir]) otherwise.
- If an algorithm's template parameter is named `NoThrowBidirectionalIterator1` or `NoThrowBidirectionalIterator2`, the template argument is also required to have the property that no exceptions are thrown from increment, decrement, assignment, or comparison of, or indirection through, valid iterators.

§ 3.14. [uninitialized.relocate]

DRAFTING NOTE: Compare to [uninitialized.move] and [alg.copy]. The *Remarks* allude to blanket wording in [specialized.algorithms.general]/2.

Add a new section after [uninitialized.move]:

uninitialized_relocate [uninitialized.relocate]

```
template<class NoThrowInputIterator, class NoThrowForwardIterator>
NoThrowForwardIterator uninitialized_relocate(NoThrowInputIterator first, NoThrowInputI
NoThrowForwardIterator result);
```

1. Effects: Equivalent to:

```
try {
    for (; first != last; ++result, (void)++first) {
        ::new (voidify(*result))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
        destroy_at(addressof(*first));
    }
    return result;
} catch (...) {
    destroy(++first, last);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable ([[basic.types.general](#)]), side effects associated with the relocation of values do not happen.

2. Remarks: If an exception is thrown, all objects in both the source and destination ranges are destroyed.

```
template<class NoThrowInputIterator, class Size, class NoThrowForwardIterator>
pair<NoThrowInputIterator, NoThrowForwardIterator>
uninitialized_relocate_n(NoThrowInputIterator first, Size n, NoThrowForwardIterator
```

3. Effects: Equivalent to:

```
try {
    for (; n > 0; ++result, (void)++first, --n) {
        ::new (voidify(*result))
            typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*first));
        destroy_at(addressof(*first));
    }
    return {first, result};
} catch (...) {
    destroy_n(++first, --n);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable ([[basic.types.general](#)]), side effects associated with the relocation of values do not happen.

4. *Remarks:* If an exception is thrown, all objects in both the source and destination ranges are destroyed.

```
template<class NoThrowBidirectionalIterator1, class NoThrowBidirectionalIterator2>
    NoThrowBidirectionalIterator2
        uninitialized_relocate_backward(NoThrowBidirectionalIterator1 first,
                                         NoThrowBidirectionalIterator1 last,
                                         NoThrowBidirectionalIterator2 result);
```

5. *Effects:* Equivalent to:

```
try {
    for (; last != first; ) {
        --last;
        --result;
        ::new (voidify(*result))
            typename iterator_traits<NoThrowBidirectionalIterator2>::value_type(std::move(*last));
        destroy_at(addressof(*last));
    }
    return result;
} catch (...) {
    destroy(first, ++last);
    throw;
}
```

except that if the iterators' common value type is trivially relocatable ([[basic.types.general](#)]), side effects associated with the relocation of values do not happen.

6. *Remarks:* If an exception is thrown, all objects in both the source and destination ranges are destroyed.

§ 3.15. [[specialized.relocate](#)]

Add a new section after [[specialized.destroy](#)]:

relocate [**specialized.relocate**]

```
template<class T>
T *relocate at(T* source, T* dest);
```

1. *Mandates*: T is a complete non-array object type.

2. *Effects*: Equivalent to:

```
struct guard { T *t; ~guard() { destroy_at(t); } } g(source);
return ::new (voidify(*dest)) T(std::move(*source));
```

except that if T is trivially relocatable ([[basic.types.general](#)]), side effects associated with the relocation of the value of *source do not happen.

```
template<class T>
remove_cv t<T> relocate(T* source);
```

3. *Mandates*: T is a complete non-array object type.

4. *Effects*: Equivalent to:

```
remove_cv t<T> t = std::move(source);
destroy_at(source);
return t;
```

except that if T is trivially relocatable ([[basic.types.general](#)]), side effects associated with the relocation of the value of *source do not happen.

§ 4. Acknowledgements

Thanks to Pablo Halpern for [[N4158](#)], to which this proposal bears a striking resemblance—including the meaning assigned to the word "trivial," and the library-algorithm approach to avoiding the problems with "lame duck objects" discussed in [the final section](#) of [[N1377](#)]. See [discussion of N4034 at Rapperswil](#) (June 2014) and [discussion of N4158 at Urbana](#) (November 2014).

Significantly different approaches to this problem have previously appeared in Rodrigo Castro Campos's [[N2754](#)], Denis Bider's [[P0023R0](#)] (introducing a core-language "relocation" operator), and Niall Douglas's [[P1029R3](#)] (treating trivial relocatability as an aspect of move-construction in isolation, rather than an aspect of the class type as a whole).

A less different approach is taken by Mungo Gill & Alisdair Meredith's [[P2786](#)]. [[P2814R1](#)] compares P2786R0 against P1144R8.

Thanks to Elias Kosunen, Niall Douglas, John Bandela, and Nicolas Lesser for their feedback on early drafts of P1144R0. Thanks to Jens Maurer for his feedback on P1144R3 at Kona 2019, and to Corentin Jabot for championing P1144R4 at Prague 2020.

Many thanks to Matt Godbolt for allowing me to install my Clang fork on Compiler Explorer (godbolt.org). See also [\[Announcing\]](#).

Thanks to Nicolas Lesser and John McCall for their review comments on the original Clang pull request [\[D50119\]](#). Thanks to Amirreza Ashouri for reviving the Clang effort in 2024 as [#84621](#), and to the dozens of GitHub users who have starred and library maintainers who have commented in support of that review.

Thanks to Howard Hinnant for appearing with me on [\[CppChat\]](#) in 2018, and to Jon Kalb and Phil Nash for hosting us.

Thanks to Marc Glisse for his work integrating a "trivially relocatable" trait into GNU libstdc++ (see [\[Deque\]](#)) and for answering my questions on [GCC bug 87106](#).

Thanks to Dana Jansens for her contributions re overlapping and non-standard-layout types (see [\[Subspace\]](#)), to Alisdair Meredith for our extensive discussions during the February 2023 drafting of P2786R0, and to Giuseppe D'Angelo for extensive review comments and discussion.

Thanks to Charles Salvia ([stdx::error](#)), Isidoros Tsaousis-Seiras (HPX), Orvid King (Folly), Daniel Anderson (Parlay), and Derek Mauro (Abseil) for their work integrating P1144 into their respective libraries. Special thanks to Stéphane Janel (AMC) for doing the work completely independently, such that I only found out after the fact. :)

Thanks to Giuseppe D'Angelo for [\[P3233\]](#). Thanks to Alan de Freitas, Daniel Liam Anderson, Giuseppe D'Angelo, Hans Goudey, Hartmut Kaiser, Isidoros Tsaousis, Jacques Lucke, Krystian Stasiowski, Shreyas Atre, Stéphane Janel, and Thiago Maciera for [\[P3236\]](#).

Thanks to... you? If you can help (by adding relevant support to your third-party library; by submitting P1144-style optimizations to standard libraries; by writing a GCC patch; by writing a blog post; in any other way), please reach out!

§ [Appendix A: Straw polls](#)

§ [Poll taken in EWG at St Louis on 2024-06-28](#)

Pablo Halpern presented [\[P2786\]](#) along with a sketch of P2786's roadmap to parity with P1144 (namely [\[P2967\]](#), [\[P3239\]](#), and [\[D3262\]](#)). On the other side, Giuseppe D'Angelo presented [\[P3233\]](#)

and [\[P3236\]](#).

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>Given the new information received (P3233R0, P3236R1, P3278R0), we wish to un-forward P2786 from CWG and bring it back to EWG.</i>	<i>21</i>	<i>15</i>	<i>3</i>	<i>6</i>	<i>5</i>

§ Polls taken in EWGI at Issaquah on 2023-02-10

Arthur O'Dwyer presented [\[P1144R6\]](#). Alisdair Meredith presented [P2786R0](#) (which proposed a `[[maybe_trivially_relocatable]]`-style facility, and expressed it as a contextual keyword instead of an attribute). EWGI took the following [straw polls](#) (as well as polls on attribute syntax and on both papers' readiness for EWG).

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>The problem presented in P1144/P2786 is worth solving.</i>	<i>10</i>	<i>8</i>	<i>0</i>	<i>0</i>	<i>0</i>
<i>The problem being introduced in P1144/P2786 should be solved in a more general way instead of as proposed.</i>	<i>3</i>	<i>0</i>	<i>5</i>	<i>6</i>	<i>4</i>
<i>The annotation should "trust the user" as in P1144R6's <code>[[trivially_relocatable]]</code> ("sharp knife"), instead of diagnosing as in P1144R6's <code>[[clang::maybe_trivially_relocatable]]</code> and P2786R0's <code>trivially_relocatable</code> ("dull knife"). Three-way poll.</i>	<i>—</i>	<i>7</i>	<i>5</i>	<i>6</i>	<i>—</i>
<i>Forward P1144 to EWG.</i>	<i>0</i>	<i>7</i>	<i>4</i>	<i>3</i>	<i>1</i>

§ Polls taken in EWGI at Prague on 2020-02-13

Corentin Jabot championed P1144R4. EWGI discussed P1144R4 and Niall Douglas's [\[P1029R3\]](#) consecutively, then took the following [straw polls](#) (as well as a poll on the attribute syntax).

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>We believe that P1029 and P1144 are sufficiently different that they should be advanced separately.</i>	<i>7</i>	<i>3</i>	<i>2</i>	<i>0</i>	<i>0</i>

<i>EWGI is ok to have the spelling as an attribute with an expression argument.</i>	3	5	1	1	0
<i>EWGI thinks the author should explore P1144 as a customizable type trait.</i>	0	0	0	9	2
<i>Forward P1144 to EWG.</i>	1	3	4	1	0

For polls taken September–November 2018, see [P1144R6].

§ References

§ Informative References

[Announcing]

Arthur O'Dwyer. *Announcing "trivially relocatable"*. July 2018. URL: <https://quuxplusone.github.io/blog/2018/07/18/announcing-trivially-relocatable/>

[CppChat]

Howard Hinnant; Arthur O'Dwyer. *cpp.chat episode 40: It works but it's undefined behavior*. August 2018. URL: <https://www.youtube.com/watch?v=8u5Qi4FgTP8>

[D3262]

Alisdair Meredith. *Specifying Trivially Relocatable Types in the Standard Library*. unreleased (May 2024). URL: <https://isocpp.org/files/papers/D3262R0.pdf>

[D50119]

Arthur O'Dwyer; Nicolas Lesser; John McCall. *Compiler support for P1144R0 `__is_trivially_relocatable(T)`*. July 2018. URL: <https://reviews.llvm.org/D50119>

[Deque]

Marc Glisse. *Improve relocation ... (`__is_trivially_relocatable`): Specialize for deque*. November 2018. URL: <https://github.com/gcc-mirror/gcc/commit/a9b9381580de611126c9888c1a6c12a77d9b682e>

[N1377]

Howard Hinnant; Peter Dimov; Dave Abrahams. *N1377: A Proposal to Add Move Semantics Support to the C++ Language*. September 2002. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1377.htm>

[N2754]

Rodrigo Castro Campos. *N2754: TriviallyDestructibleAfterMove and TriviallyReallocatable (rev 3)*. September 2008. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2008/n2754.html>

[N4158]

Pablo Halpern. *N4158: Destructive Move (rev 1)*. October 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf>

[P0023R0]

Denis Bider. *P0023R0: Relocator: Efficiently Moving Objects*. April 2016. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0023r0.pdf>

[P1029R3]

Niall Douglas. *P1029R3: move = bitcopies*. January 2020. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1029r3.pdf>

[P1144R6]

Arthur O'Dwyer. *Object relocation in terms of move plus destroy*. June 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1144r6.html>

[P2786]

Mungo Gill; Alisdair Meredith. *Trivial Relocatability For C++26*. September 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2786r7.pdf>

[P2814R1]

Mungo Gill; Alisdair Meredith. *Trivial relocatability — comparing P2786 with P1144*. May 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2814r1.pdf>

[P2967]

Alisdair Meredith; Mungo Gill. *Relocation has a library interface*. May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2967r1.pdf>

[P3233]

Giuseppe D'Angelo. *Issues with P2786 ('Trivial Relocatability For C++26')*. April 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3233r0.html>

[P3236]

Alan de Freitas; et al. *Please reject P2786 and adopt P1144*. May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3236r1.html>

[P3239]

Alisdair Meredith. *A relocating swap*. May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3239r0.pdf>

[P3279]

Arthur O'Dwyer. *CWG2463: What 'trivially fooable' should mean*. May 2024. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3279r0.html>

[Polymorphic]

Arthur O'Dwyer. *Polymorphic types aren't trivially relocatable*. June 2023. URL: <https://quuxplu-sone.github.io/blog/2023/06/24/polymorphic-types-arent-trivially-relocatable/>

[Subspace]

Dana Jansens. *Trivially Relocatable Types in C++/Subspace*. January 2023. URL:
<https://danakj.github.io/2023/01/15/trivially-relocatable.html>